

Práctica 1

Guion 1

Búsqueda Local y Trayectorias: El Viajante de Comercio (TSP)

Metaheurísticas (Curso 2026/2027)

Algoritmos implementados:

Greedy · Greedy aleatorio · Búsqueda local del primer mejor (DLB) · Búsqueda tabú

Autores:

Alejandro Ríos Cabello	arc00166	DNI: 77958591E
Nicolás Rafael Chelaru Tanase	nrct0001	DNI: Y1658743C

Grupo: XX

Índice

1 Descripción del problema **3**
1.1 El problema del viajante de comercio 3
1.2 Representación de las soluciones 3
1.3 Función objetivo 3
1.4 Operadores comunes 3
1.5 Descripción del código 4

2 Algoritmos **4**
2.1 Greedy 4
2.2 Greedy aleatorio 5
2.3 Búsqueda local del primer mejor con DLB 5
2.4 Búsqueda tabú 6

3 Experimentos y análisis de resultados **10**
3.1 Parámetros y semillas 10
3.2 Resultados 10
3.3 Análisis de los algoritmos 13
3.4 Tiempos, desviaciones e influencia del tamaño 17
3.5 Análisis de la mejor solución 18

Anexo **19**

A Carpeta de Logs **19**

Índice de Figuras

1 Diagrama de flujo de la búsqueda tabú. 9
2 Desviación media respecto al óptimo por algoritmo e instancia (escala logarítmica). 13
3 Convergencia de la mejor ejecución de la búsqueda tabú y de la ejecución de la búsqueda local con la misma semilla en ch130, a280 y u1060 (coste en escala logarítmica). La línea fina es la solución actual del tabú y la discontinua, el óptimo. 16
4 Tiempo medio por ejecución frente al número de ciudades (escalas logarítmicas). 17

Índice de Tablas

1 Parámetros de las ejecuciones. 10
2 Resultados de Greedy 11
3 Resultados de Greedy aleatorio 11
4 Resultados de Búsqueda local 11
5 Resultados de Búsqueda tabú 12
6 Desviación media respecto al óptimo y tiempo medio por instancia. GRE: greedy; GRA: greedy aleatorio; BL: búsqueda local; TA: búsqueda tabú. En negrita, la menor desviación de cada instancia. 12
7 Movimientos realizados por la búsqueda local y motivo de parada. 14
8 Comportamiento interno de la búsqueda tabú (10.000 iteraciones por ejecución). 15

1. Descripción del problema

1.1. El problema del viajante de comercio

El problema del viajante de comercio (TSP, *Travelling Salesman Problem*) consiste en encontrar, dado un conjunto de n ciudades y la distancia entre cada par de ellas, el recorrido cerrado más corto que pasa por todas las ciudades exactamente una vez y vuelve a la de origen. Se trata de un problema NP-duro. El número de recorridos distintos crece como $(n-1)!/2$, así que con las instancias que usamos (de 130 a 18.512 ciudades) no es viable una búsqueda exhaustiva y hay que recurrir a metaheurísticas.

Las instancias son de TSPLIB (`ch130`, `pr144`, `a280`, `u1060` y `d18512`). En cada fichero vienen las coordenadas (x_i, y_i) de las ciudades. El problema es simétrico y la distancia entre dos ciudades es la euclídea,

$$d(i,j) = d(j,i) = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}. \tag{1}$$

Al leer el fichero calculamos una sola vez la matriz de distancias D, de tamaño $n \times n$, y así consultar una distancia durante la búsqueda cuesta $O(1)$.

1.2. Representación de las soluciones

Representamos cada solución como una permutación $\pi = (\pi_0, \pi_1, \ldots, \pi_{n-1})$ de las ciudades $\{0, \ldots, n-1\}$, donde π_k es la ciudad que se visita en la posición k del recorrido. Después de la última ciudad se vuelve a la primera, por lo que el recorrido es circular. Con esta representación cada ciudad aparece una sola vez por construcción, y todos los operadores que usamos transforman una permutación en otra.

1.3. Función objetivo

El objetivo es minimizar la longitud total del recorrido,

$$F(\pi) = \sum_{k=0}^{n-1} d\left(\pi_k, \pi_{(k+1) \bmod n}\right). \tag{2}$$

Evaluar una solución completa tiene coste $O(n)$. Para medir la calidad usamos la desviación respecto al óptimo conocido de cada instancia,

$$\mathrm{Desv}(\pi) = 100 \cdot \frac{F(\pi) - F^*}{F^*}. \tag{3}$$

1.4. Operadores comunes

Operador de intercambio (2-opt del guion). Dadas dos posiciones $i \neq j$, se intercambian las ciudades que hay en ellas ($\pi_i \leftrightarrow \pi_j$). Con este operador cada solución tiene $n(n-1)/2$ vecinos.

Evaluación factorizada. Cuando se intercambian dos posiciones solo cambian los arcos que llegan a ellas o salen de ellas, así que el coste del vecino se puede calcular en $O(1)$ a partir del coste actual:

$$F(\pi') = F(\pi) - \text{arcos que desaparecen} + \text{arcos nuevos}. \tag{4}$$

Si i y j no son adyacentes, desaparecen los cuatro arcos (π_{i-1}, π_i), (π_i, π_{i+1}), (π_{j-1}, π_j), (π_j, π_{j+1}) y aparecen otros cuatro que unen esos vecinos con la ciudad intercambiada. Si son adyacentes (por ejemplo, $j = i + 1$), el arco (π_i, π_j) se mantiene aunque invertido, y solo cambian los dos arcos exteriores. Los índices se toman módulo n. De esta forma la primera y la última posición, que en un recorrido circular también son adyacentes, se tratan igual que cualquier otro par de posiciones contiguas.

Don't Look Bits (DLB). Es un vector de n bits, uno por posición. Si el bit de una posición está a 0, merece la pena explorar sus intercambios, y si está a 1 es que ya se exploraron todos y ninguno mejoraba. Al principio todos valen 0. Cuando una posición agota sus movimientos sin encontrar mejora su bit pasa a 1, y vuelve a 0 si la posición participa en un movimiento que se aplica. Con esto la búsqueda se centra en las partes del recorrido que han cambiado hace poco.

Generación de semillas. Partimos de la semilla base `1658743` y en cada ejecución rotamos la anterior un dígito a la izquierda, así que las tres ejecuciones usan las semillas 6587431, 5874316 y 8743165. La rotación se reinicia desde la semilla base en cada algoritmo y en cada instancia, de modo que la ejecución k usa la misma semilla en todos los algoritmos, tal y como pide el guion. Con la semilla se inicializa el generador `random.Random` que usan el greedy aleatorio, la búsqueda local y la búsqueda tabú, y así cualquier ejecución se puede reproducir.

1.5. Descripción del código

El código está hecho en Python y lo hemos dividido en estos módulos:

- `main.py`: lee `parametros.txt`, carga cada instancia y lanza los algoritmos indicados en `ALGORITMO`, una vez por semilla.
- `extracion_datos.py`: lee el fichero `.tsp` y calcula la matriz de distancias con NumPy.
- `Extraccion_parametros.py`: lee los parámetros y genera las semillas por rotación.
- `Funcion_evaluacion.py`: función objetivo, evaluación factorizada del intercambio y aplicación del movimiento.
- `greedy.py`, `greedy_aleatorio.py`, `Busqueda_local.py` y `Busqueda_Tabu.py`: un módulo por algoritmo.
- `logs.py`: escribe los ficheros de registro. Los movimientos se van guardando en memoria y se escriben por bloques. El tiempo que se pierde escribiendo se resta del tiempo de ejecución que damos en los resultados.

2. Algoritmos

2.1. Greedy

Para cada ciudad calculamos la suma de sus distancias al resto, ordenamos el vector de pares ⟨ciudad, suma⟩ de menor a mayor y tomamos ese orden como recorrido. Es un algoritmo determinista y su coste es $O(n^2 + n \log n)$.

Algoritmo 1 Greedy

Entrada: Matriz de distancias D de tamaño n
Salida: Recorrido π

1: $V \leftarrow [\,]$
2: **para** $i \leftarrow 0$ **hasta** $n-1$ **hacer**
3: añadir $\langle i, \sum_j D[i][j] \rangle$ a V
4: **fin para**
5: ordenar V por la suma de forma ascendente
6: $\pi \leftarrow$ ciudades de V en ese orden
7: **devolver** π

2.2. Greedy aleatorio

Usa el mismo vector ordenado, pero en cada paso escoge al azar una de las $K = 5$ primeras ciudades que quedan, la añade al recorrido y la quita del vector. Cuando quedan menos de K ciudades se escoge entre las que haya.

Algoritmo 2 Greedy aleatorio

Entrada: D, generador aleatorio, K
Salida: Recorrido π

1: $V \leftarrow$ vector $\langle$ciudad, suma de distancias$\rangle$ ordenado ascendentemente
2: $\pi \leftarrow [\,]$
3: **mientras** V no está vacío **hacer**
4: $r \leftarrow$ entero aleatorio en $[0, \text{mín}(K, |V|) - 1]$
5: añadir la ciudad $V[r]$ a π y eliminarla de V
6: **fin mientras**
7: **devolver** π

2.3. Búsqueda local del primer mejor con DLB

La búsqueda local parte de una solución del greedy aleatorio. Va recorriendo las posiciones cuyo bit DLB está a 0 y, para cada una, prueba a intercambiarla con el resto de posiciones (de forma circular, empezando en $i + 1$). En cuanto encuentra un intercambio que mejora el coste lo aplica (primer mejor), reactiva los bits de las dos posiciones y vuelve a explorar la misma posición. Si no hay ninguno que mejore, desactiva su bit. El algoritmo termina cuando todos los bits están a 1, es decir, cuando se ha llegado a un óptimo local para el operador de intercambio, o cuando se alcanzan los 10.000 movimientos.

Algoritmo 3 Búsqueda local del primer mejor con DLB

Entrada: D, máximo de iteraciones M, K
Salida: Mejor recorrido encontrado

```
1: π ← GreedyAleatorio(D, K);    c ← F(π)
2: dlb ← [0, ..., 0];    it ← 0;    i ← 0
3: mientras it < M y algún dlb[i] = 0 hacer
4:     si dlb[i] = 0 entonces
5:         mejora ← falso
6:         para k ← 1 hasta n − 1 hacer
7:             j ← (i + k) mód n
8:             si Factorizacion(π, i, j, c) < c entonces
9:                 intercambiar π_i y π_j;    c ← F(π);    it ← it + 1
10:                dlb[i] ← 0;    dlb[j] ← 0;    mejora ← verdadero
11:                salir del bucle                ▷ se vuelve a explorar la misma i
12:            fin si
13:        fin para
14:        si no mejora entonces
15:            dlb[i] ← 1;    i ← (i + 1) mód n
16:        fin si
17:    si no
18:        i ← (i + 1) mód n
19:    fin si
20: fin mientras
21: devolver π
```

2.4. Búsqueda tabú

La búsqueda tabú extiende la búsqueda local para que no se pare en el primer óptimo local. Siempre se ejecutan las 10.000 iteraciones y, cuando ningún vecino permitido mejora la solución actual, se aplica el mejor vecino no tabú aunque empeore el coste ("la mejor de las peores"). Durante la búsqueda guardamos tres soluciones, que son la solución actual, el mejor del momento actual (la mejor desde el último reinicio) y el mejor global.

Memorias. Para las dos memorias usamos una sola matriz T de enteros de tamaño $n \times n$.

- **Memoria a corto plazo (MCP)**, en el triángulo superior ($T[a][b]$ con $a < b$). Cuando se intercambian las posiciones i y j, que contienen las ciudades π_i y π_j, se guardan dos valores con la tenencia tabú $t = 10$. Para el par de posiciones se hace $T[\text{mín}(i,j)][\text{máx}(i,j)] \leftarrow t$ y para el par de ciudades se hace $T[\text{mín}(\pi_i,\pi_j)][\text{máx}(\pi_i,\pi_j)] \leftarrow t$. Un movimiento es tabú si cualquiera de sus dos celdas es mayor que 0, y en ese caso se descarta al generar el entorno. No hemos usado criterio de aspiración. Para no tener que recorrer la matriz entera en cada iteración guardamos las celdas activas en un conjunto, y en cada iteración solo se decrementan esas celdas y se quitan las que llegan a 0.
- **Memoria a largo plazo (MLP)**, en el triángulo inferior ($T[a][b]$ con $a > b$). En cada iteración se suma 1 a la celda de cada arco (π_k, π_{k+1}) de la solución actual, o

sea, n celdas por iteración. Al ser un problema simétrico, los arcos (u, v) y (v, u) comparten celda.

Exploración del entorno. Se explora igual que en la búsqueda local (DLB y primer mejor), pero saltándose los movimientos tabú. Mientras se recorren los vecinos se va guardando el mejor vecino no tabú que ha aparecido. Si se encuentra una mejora se aplica, y si todos los bits del DLB acaban a 1 sin ninguna mejora se aplica ese mejor vecino. Después de aplicar un movimiento, sea del tipo que sea, solo se reactivan los bits de las dos posiciones intercambiadas. Por eso la siguiente exploración evalúa unos $2n$ vecinos y no los $n^2/2$ del entorno completo.

Estancamiento y oscilación estratégica. Contamos como movimiento de empeoramiento cualquier movimiento que no mejore al mejor del momento actual. Cuando se acumulan $0{,}05 \cdot 10.000 = 500$ seguidos, consideramos que la búsqueda se ha estancado y generamos una solución nueva, por intensificación con probabilidad 0,5 o por diversificación con probabilidad 0,5. Los dos operadores construyen un recorrido empezando en una ciudad aleatoria. En cada paso eligen al azar entre las $K = 5$ ciudades no visitadas cuyo arco con la ciudad actual tiene mayor frecuencia en la MLP, en el caso de la intensificación, o menor frecuencia, en el de la diversificación. La intensificación reconstruye así los arcos que más han aparecido en las buenas soluciones, mientras que la diversificación lleva la búsqueda a zonas poco visitadas. Después se reinician la MCP, el DLB, el contador de estancamiento y el mejor del momento actual. La MLP se conserva.

Algoritmo 4 Búsqueda tabú

Entrada: D, $M = 10.000$, $K = 5$, tenencia $t = 10$, estancamiento $e = 0{,}05$, oscilación $p = 0{,}5$

Salida: Mejor recorrido global

$\pi \leftarrow$ GreedyAleatorio(D, K); $\pi_{mom} \leftarrow \pi$; $\pi_{glob} \leftarrow \pi$
$T \leftarrow$ matriz $n \times n$ de ceros; $dlb \leftarrow [0, \ldots, 0]$; $it \leftarrow 0$; $est \leftarrow 0$
mientras $it < M$ **hacer**
    $(i, j) \leftarrow$ primer movimiento no tabú que mejora π usando el DLB
    si no existe **entonces**
        $(i, j) \leftarrow$ mejor movimiento no tabú evaluado ▷ *la mejor de las peores*
    fin si
    si todos los movimientos evaluados son tabú **entonces**
        $est \leftarrow e \cdot M$ ▷ *se fuerza el reinicio*
    si no
        decrementar en 1 las celdas activas de la MCP
        T[posiciones i, j] $\leftarrow t$; T[ciudades π_i, π_j] $\leftarrow t$
        intercambiar π_i y π_j; $dlb[i] \leftarrow 0$; $dlb[j] \leftarrow 0$; $it \leftarrow it + 1$
        incrementar en la MLP los n arcos de π
        si $F(\pi) < F(\pi_{mom})$ **entonces**
            $\pi_{mom} \leftarrow \pi$; $est \leftarrow 0$
            si $F(\pi) < F(\pi_{glob})$ **entonces** $\pi_{glob} \leftarrow \pi$
            fin si
        si no
            $est \leftarrow est + 1$
        fin si
    fin si
    si $est \geq e \cdot M$ **entonces**
        si aleatorio $< p$ **entonces**
            $\pi \leftarrow$ Intensificacion(T, K)
        si no
            $\pi \leftarrow$ Diversificacion(T, K)
        fin si
        $\pi_{mom} \leftarrow \pi$; actualizar π_{glob} si π lo mejora
        reiniciar MCP, dlb y est ▷ *la MLP no se reinicia*
    fin si
fin mientras
devolver π_{glob}

En la Figura 1 se muestra el mismo proceso en forma de diagrama de flujo.

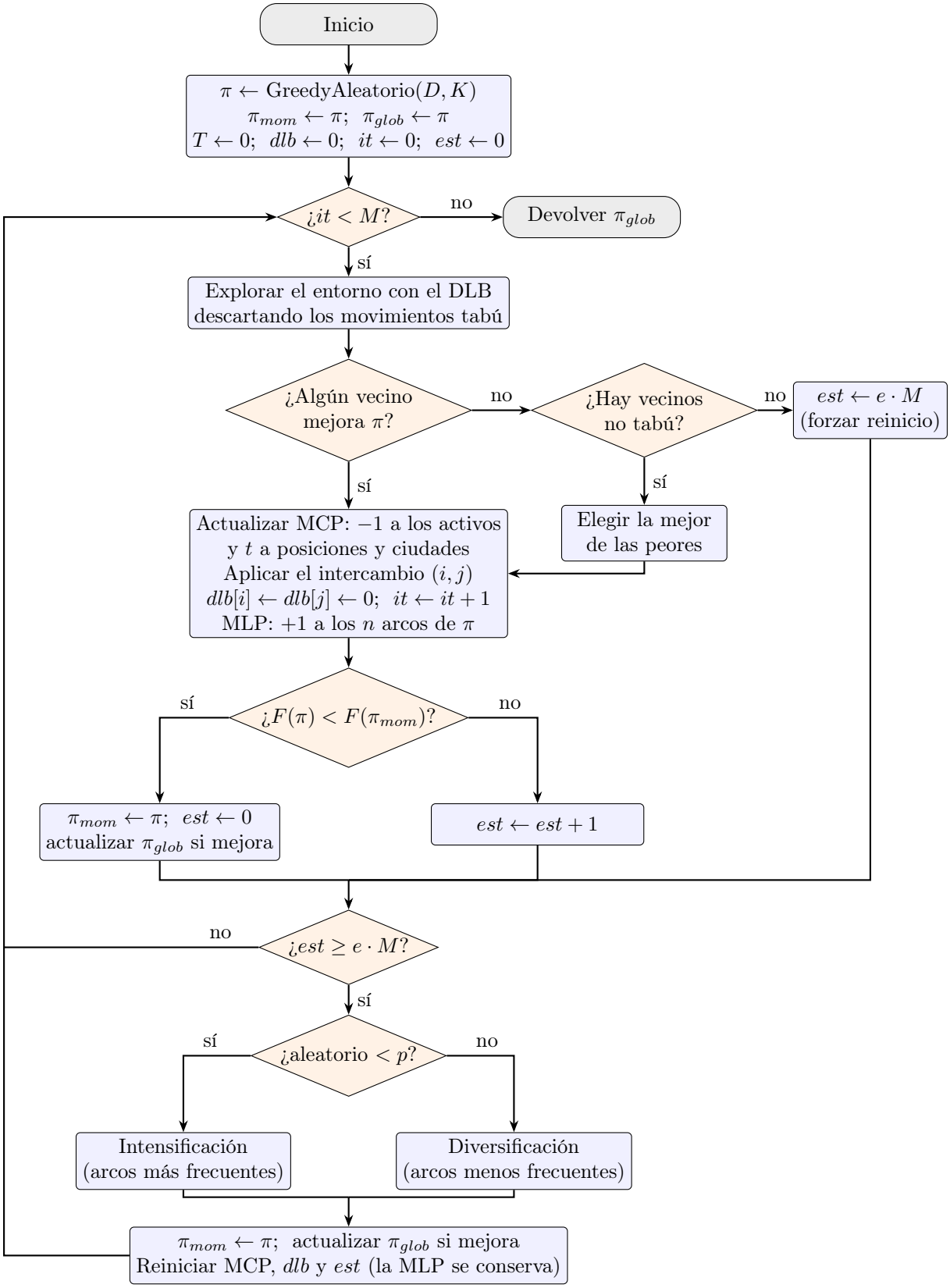


Figura 1: Diagrama de flujo de la búsqueda tabú.

Algoritmo 5 Construcción de una solución a partir de la MLP (intensificación y diversificación)

Entrada: MLP, K, modo $\in$ {intensificación, diversificación}
Salida: Recorrido π

```
actual ← ciudad aleatoria;   π ← [actual]
mientras |π| < n hacer
    f ← frecuencias en la MLP de los arcos (actual, v) para cada v no visitada
    si modo = intensificación entonces
        C ← las K ciudades no visitadas con mayor f
    si no
        C ← las K ciudades no visitadas con menor f
    fin si
    actual ← ciudad aleatoria de C;   añadir actual a π
fin mientras
devolver π
```

3. Experimentos y análisis de resultados

3.1. Parámetros y semillas

Todos los parámetros se leen del fichero `parametros.txt`.

Parámetro	Valor	Algoritmos
Semillas	6587431, 5874316, 8743165	GA, BL, Tabú
Ejecuciones por instancia	3 (1 en el greedy, que es determinista)	Todos
K	5	GA, BL, Tabú
Máximo de iteraciones	10.000	BL, Tabú
Tenencia tabú	10	Tabú
Estancamiento	5 % de las iteraciones (500)	Tabú
Oscilación estratégica	50 %	Tabú

Tabla 1: Parámetros de las ejecuciones.

En todos los algoritmos y en todas las instancias la ejecución k usa la misma semilla. Así, en cada ejecución la búsqueda local y la búsqueda tabú parten de la misma solución del greedy aleatorio y sus resultados se pueden comparar directamente. Los tiempos se han medido todos en el mismo equipo y no incluyen la escritura de los logs.

3.2. Resultados

En las Tablas 2 a 5 aparecen el coste y el tiempo de cada ejecución, además del mejor y el peor valor, la media y la desviación típica. La Tabla 6 resume la desviación media respecto al óptimo y el tiempo medio. Como óptimos hemos tomado ch130 = 6.110, pr144 = 58.537, a280 = 2.579, u1060 = 224.094 y d18512 = 645.238.

Instancia	Coste	Desv. (%)	Tiempo (s)
ch130	40.397,47	561,17	< 0,001
pr144	640.770,87	994,64	< 0,001
a280	30.296,82	1.074,75	0,001
u1060	5.670.064,52	2.430,22	0,003
d18512	51.161.687,44	7.829,12	0,171

Tabla 2: Resultados de Greedy (determinista: una sola ejecución por instancia).

		Ejecución			Estadísticos			
Instancia		1	2	3	Mejor	Peor	Media	Desv. típ.
ch130	Coste	41.107	41.379	40.198	40.198	41.379	40.894	618
	Tiempo (s)	0,001	0,001	0,001	0,001	0,001	0,001	< 0,001
pr144	Coste	644.001	748.053	688.917	644.001	748.053	693.657	52.188
	Tiempo (s)	0,001	0,001	0,001	0,001	0,001	0,001	< 0,001
a280	Coste	29.923	29.455	30.283	29.455	30.283	29.887	415
	Tiempo (s)	0,002	0,002	0,002	0,002	0,002	0,002	< 0,001
u1060	Coste	5.432.361	5.726.073	5.611.772	5.432.361	5.726.073	5.590.068	148.054
	Tiempo (s)	0,020	0,020	0,020	0,020	0,020	0,020	< 0,001
d18512	Coste	51.931.042	51.597.814	50.934.175	50.934.175	51.931.042	51.487.677	507.478
	Tiempo (s)	7,354	8,061	7,389	7,354	8,061	7,601	0,398

Tabla 3: Resultados de Greedy aleatorio: coste y tiempo de cada ejecución (semillas 6587431, 5874316, 8743165, iguales en todas las instancias). Mejor y peor: menor y mayor valor de la fila. Costes redondeados a la unidad.

		Ejecución			Estadísticos			
Instancia		1	2	3	Mejor	Peor	Media	Desv. típ.
ch130	Coste	12.192	12.097	13.868	12.097	13.868	12.719	996
	Tiempo (s)	0,089	0,104	0,090	0,089	0,104	0,094	0,008
pr144	Coste	211.455	175.230	182.305	175.230	211.455	189.663	19.201
	Tiempo (s)	0,279	0,263	0,213	0,213	0,279	0,252	0,034
a280	Coste	6.093	7.488	6.848	6.093	7.488	6.810	699
	Tiempo (s)	0,649	0,697	0,704	0,649	0,704	0,684	0,030
u1060	Coste	780.085	881.699	804.383	780.085	881.699	822.056	53.062
	Tiempo (s)	12,827	12,272	12,492	12,272	12,827	12,530	0,280
d18512	Coste	51.170.029	50.828.026	50.131.543	50.131.543	51.170.029	50.709.866	529.230
	Tiempo (s)	84,423	82,503	79,828	79,828	84,423	82,251	2,308

Tabla 4: Resultados de Búsqueda local: coste y tiempo de cada ejecución (semillas 6587431, 5874316, 8743165, iguales en todas las instancias). Mejor y peor: menor y mayor valor de la fila. Costes redondeados a la unidad.

		Ejecución			Estadísticos			
Instancia		1	2	3	Mejor	Peor	Media	Desv. típ.
ch130	Coste	7.791	8.332	8.661	7.791	8.661	8.261	439
	Tiempo (s)	4,500	4,265	4,845	4,265	4,845	4,537	0,291
pr144	Coste	117.268	136.695	120.249	117.268	136.695	124.738	10.463
	Tiempo (s)	4,837	5,228	4,420	4,420	5,228	4,828	0,404
a280	Coste	4.954	4.991	4.404	4.404	4.991	4.783	329
	Tiempo (s)	9,453	9,598	10,232	9,453	10,232	9,761	0,414
u1060	Coste	815.905	852.455	730.441	730.441	852.455	799.600	62.620
	Tiempo (s)	14,978	14,468	16,725	14,468	16,725	15,390	1,184
d18512	Coste	51.140.425	50.822.609	50.163.360	50.163.360	51.140.425	50.708.798	498.376
	Tiempo (s)	91,294	87,281	92,353	87,281	92,353	90,309	2,675

Tabla 5: Resultados de Búsqueda tabú: coste y tiempo de cada ejecución (semillas 6587431, 5874316, 8743165, iguales en todas las instancias). Mejor y peor: menor y mayor valor de la fila. Costes redondeados a la unidad.

	Desviación media (%)				Tiempo medio (s)			
Instancia (n)	GRE	GRA	BL	TA	GRE	GRA	BL	TA
ch130 (130)	561,17	569,30	108,17	**35,21**	$<$ 0,001	0,001	0,094	4,537
pr144 (144)	994,64	1.084,99	224,01	**113,09**	$<$ 0,001	0,001	0,252	4,828
a280 (280)	1.074,75	1.058,86	164,04	**85,46**	0,001	0,002	0,684	9,761
u1060 (1.060)	2.430,22	2.394,52	266,84	**256,81**	0,003	0,020	12,530	15,390
d18512 (18.512)	7.829,12	7.879,64	7.759,09	**7.758,93**	0,171	7,601	82,251	90,309
Media	2.577,98	2.597,46	1.704,43	1.649,90	0,035	1,525	19,162	24,965

Tabla 6: Desviación media respecto al óptimo y tiempo medio por instancia. GRE: greedy; GRA: greedy aleatorio; BL: búsqueda local; TA: búsqueda tabú. En negrita, la menor desviación de cada instancia.

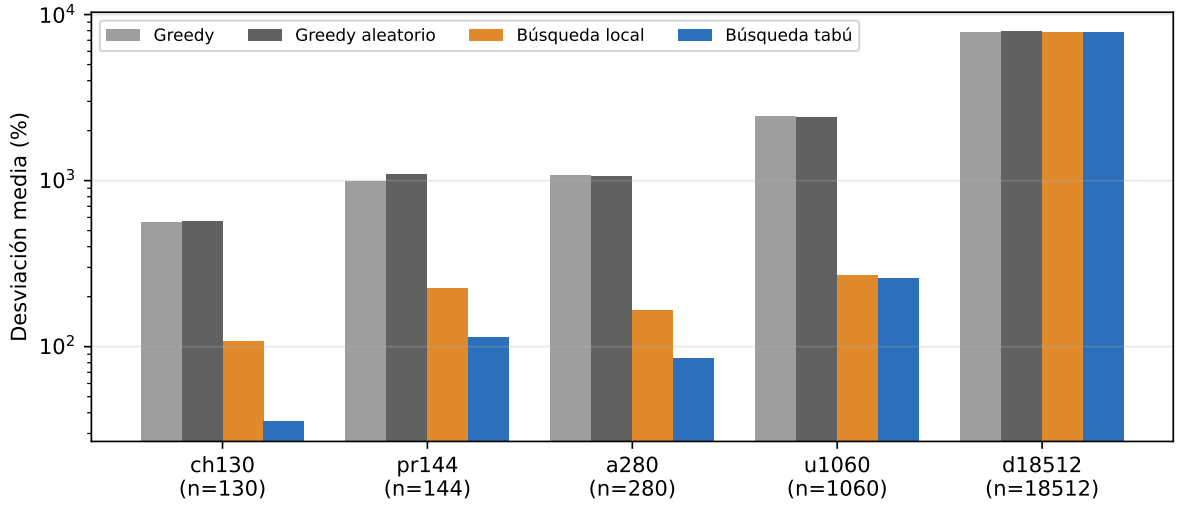


Figura 2: Desviación media respecto al óptimo por algoritmo e instancia (escala logarítmica).

3.3. Análisis de los algoritmos

Greedy y greedy aleatorio. Los dos dan recorridos entre 6 y 80 veces más largos que el óptimo (Tabla 6). El problema está en el criterio de construcción. Al ordenar las ciudades por la suma de sus distancias al resto quedan primero las ciudades centrales y después las periféricas, pero en ningún momento se mira si dos ciudades consecutivas del recorrido están cerca. Por eso el recorrido va saltando de un extremo a otro del plano. Que el greedy aleatorio elija al azar entre las $K = 5$ siguientes no arregla esto. Su media queda muy cerca de la del greedy (entre un 1 % mejor y un 8 % peor según la instancia) y lo único que añade es dispersión. Lo que tiene de útil es que es instantáneo y que da soluciones iniciales distintas para las búsquedas basadas en trayectorias.

Búsqueda local. En las cuatro instancias menores reduce la desviación del greedy aleatorio entre 4,8 y 9 veces. En la Tabla 7 se ve por qué es tan rápida en las pequeñas. En ch130, pr144 y a280 llega a un óptimo local después de unos 500–1.600 movimientos y para en menos de un segundo, ya que ningún intercambio mejora la solución. Ese óptimo local sigue estando lejos del global (98–261 % de desviación) porque el operador de intercambio es poco potente para el TSP. Al mover dos ciudades cambian cuatro arcos a la vez y casi nunca se deshace un cruce entre aristas, que es el defecto típico de un recorrido malo .

Instancia	Ejec.	Movimientos	Motivo de parada
ch130	1	551	no se encontraron más movimientos de mejora
ch130	2	575	no se encontraron más movimientos de mejora
ch130	3	493	no se encontraron más movimientos de mejora
pr144	1	622	no se encontraron más movimientos de mejora
pr144	2	659	no se encontraron más movimientos de mejora
pr144	3	617	no se encontraron más movimientos de mejora
a280	1	1.634	no se encontraron más movimientos de mejora
a280	2	1.550	no se encontraron más movimientos de mejora
a280	3	1.509	no se encontraron más movimientos de mejora
u1060	1	9.866	no se encontraron más movimientos de mejora
u1060	2	9.595	no se encontraron más movimientos de mejora
u1060	3	10.000	alcanzado el máximo de movimientos
d18512	1	10.000	alcanzado el máximo de movimientos
d18512	2	10.000	alcanzado el máximo de movimientos
d18512	3	10.000	alcanzado el máximo de movimientos

Tabla 7: Movimientos realizados por la búsqueda local y motivo de parada.

Búsqueda tabú. Es el mejor algoritmo en ch130, pr144 y a280, con una desviación media entre 1,9 y 3,1 veces menor que la de la búsqueda local. El motivo se ve en la Tabla 8, porque en estas instancias más o menos un tercio de las 10.000 iteraciones son movimientos de empeoramiento. Como acepta "la mejor de las peores" y prohíbe deshacer el movimiento durante 10 iteraciones, la búsqueda consigue salir del óptimo local en el que se habría quedado la búsqueda local y sigue bajando por otra zona. En 6 de las 9 ejecuciones la mejor solución global aparece después de la iteración 7.000, mucho después de que la búsqueda local hubiera terminado.

En ch130 las dos ejecuciones que aparecen usan la misma semilla y parten de la misma solución, así que sus curvas coinciden mientras las dos bajan. La búsqueda local se para hacia la iteración 550 y el tabú sigue mejorando poco a poco. Los picos de la solución actual son los reinicios por estancamiento (Tabla 8). Las soluciones que salen de la diversificación empiezan muy lejos del mejor coste, porque construyen el recorrido con los arcos menos usados, y a partir de ellas la búsqueda vuelve a bajar. Los reinicios por intensificación empiezan en costes mucho más bajos al reutilizar los arcos que más se han repetido. En ch130 sus soluciones iniciales cuestan entre 20.550 y 22.480, mientras que las de diversificación cuestan 50.520–55.200.

Instancia	Ejec.	Empeoramientos	Reinicios (I/D)	Última mejora global	Coste final
ch130	1	3.607	3 (1/2)	7.354	7.791,07
ch130	2	3.301	4 (1/3)	8.765	8.332,04
ch130	3	3.491	3 (0/3)	2.813	8.660,89
pr144	1	3.376	4 (2/2)	9.006	117.268,02
pr144	2	3.331	4 (3/1)	1.394	136.695,45
pr144	3	2.910	6 (1/5)	8.135	120.249,19
a280	1	2.988	3 (2/1)	9.407	4.954,24
a280	2	3.415	1 (1/0)	4.767	4.990,78
a280	3	3.691	1 (1/0)	7.345	4.404,22
u1060	1	69	0 (0/0)	10.000	815.904,79
u1060	2	0	0 (0/0)	10.000	852.455,12
u1060	3	175	0 (0/0)	9.991	730.441,48
d18512	1	0	0 (0/0)	10.000	51.140.424,59
d18512	2	0	0 (0/0)	10.000	50.822.608,96
d18512	3	0	0 (0/0)	10.000	50.163.359,62

Tabla 8: Comportamiento interno de la búsqueda tabú (10.000 iteraciones por ejecución).

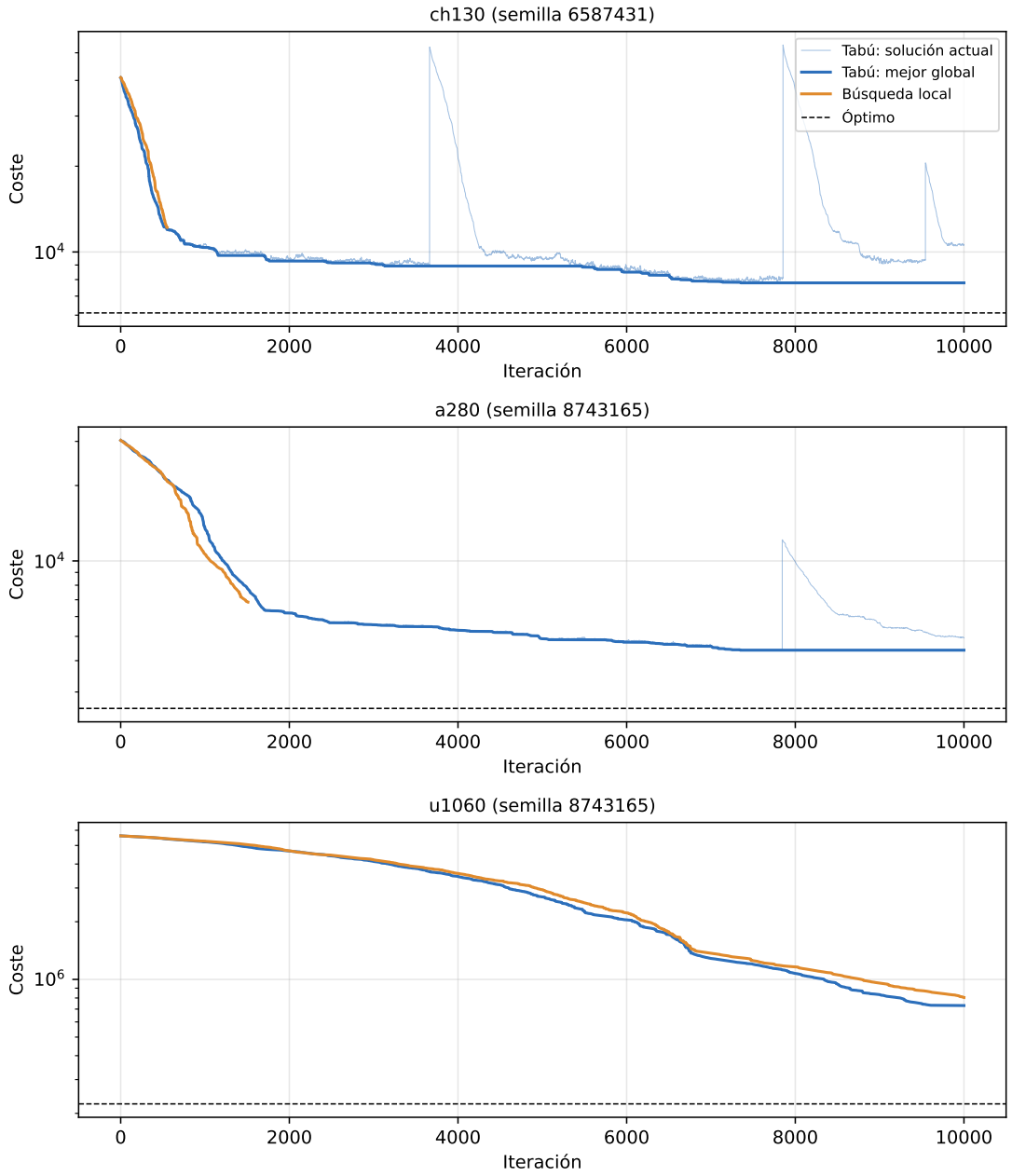


Figura 3: Convergencia de la mejor ejecución de la búsqueda tabú y de la ejecución de la búsqueda local con la misma semilla en ch130, a280 y u1060 (coste en escala logarítmica). La línea fina es la solución actual del tabú y la discontinua, el óptimo.

En u1060 el comportamiento es distinto. La búsqueda local necesita entre 9.595 y 9.866 movimientos para llegar a un óptimo local, y en la tercera ejecución se corta en el máximo de 10.000 sin llegar a él. Con el mismo presupuesto, el tabú pasa casi todas sus iteraciones bajando igual que la búsqueda local (entre 0 y 175 empeoramientos y ningún reinicio), por lo que sus mecanismos casi no llegan a actuar. En media queda algo por delante (256,8 % frente a 266,8 %), aunque la ventaja no es sistemática. Con la misma semilla, el tabú consigue un recorrido mejor en dos ejecuciones (852.455 frente a 881.699, y 730.441 frente a 804.383) y uno peor en la otra (815.905 frente a 780.085). Como la MCP prohíbe algunos movimientos de mejora durante 10 iteraciones y no hay criterio de aspiración, el tabú baja desde el principio por un camino distinto al de la búsqueda local.

En d18512 pasa lo mismo, pero más exagerado, ya que 10.000 iteraciones son muy pocas para 18.512 ciudades. Todas las iteraciones del tabú son de mejora (no hay empeoramientos ni reinicios), y la búsqueda tabú (7.758,9 % de desviación media) y la búsqueda local (7.759,1 %) acaban prácticamente en el mismo punto, solo un poco por debajo del greedy aleatorio del que parten.

3.4. Tiempos, desviaciones e influencia del tamaño

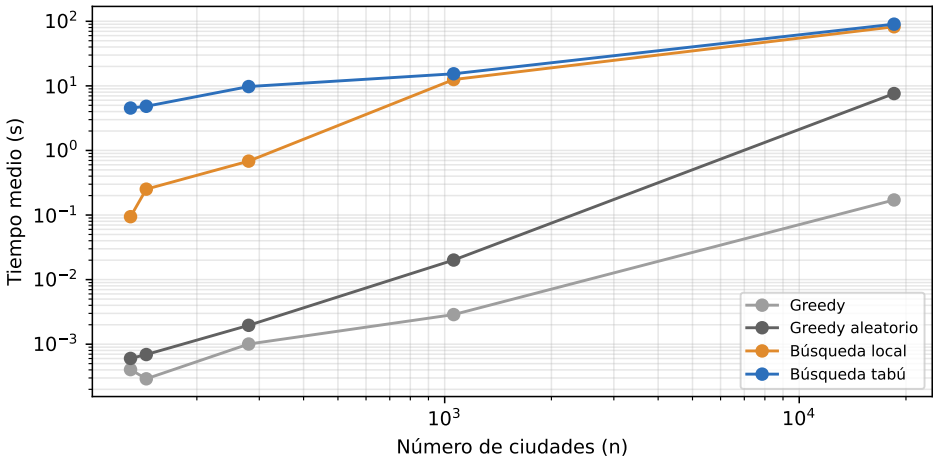


Figura 4: Tiempo medio por ejecución frente al número de ciudades (escalas logarítmicas).

El tamaño de la instancia se nota en varios aspectos.

- **Coste de los greedy.** Calcular las sumas de distancias es $O(n^2)$, pero como lo hacemos con operaciones vectorizadas de NumPy el greedy tarda 0,17 s incluso en d18512. El greedy aleatorio necesita 7,6 s en esa instancia porque vuelve a ordenar el vector de candidatos cada vez que inserta una ciudad. Solo la matriz de distancias de d18512 ya ocupa alrededor de 1,3 GB.
- **Presupuesto de iteraciones.** Los movimientos que hacen falta para llegar a un óptimo local aumentan con n (unos 540 en ch130, 1.560 en a280 y más de 9.500 en u1060). Con un límite fijo de 10.000 iteraciones, en u1060 la búsqueda local llega muy justa y en d18512 se queda muy lejos. Las tres ejecuciones de la búsqueda local y las tres del tabú se cortan en 10.000 movimientos con una desviación superior al 7.600 %, casi igual que la del greedy aleatorio del que parten. A partir de cierto tamaño, la calidad viene limitada más por el número de iteraciones permitido que por el algoritmo.
- **Tiempo por iteración.** La evaluación factorizada es $O(1)$, pero en cada iteración también hay operaciones $O(n)$, como recalcular el coste o actualizar la MLP, y una exploración del DLB que puede llegar a recorrer n vecinos por posición. Por eso el tabú, que siempre hace las 10.000 iteraciones, tarda en las instancias pequeñas entre 14 y 48 veces más que la búsqueda local, que termina antes. En u1060, donde las dos hacen casi todas las iteraciones, la diferencia baja a 1,2 veces (15,4 s frente a 12,5 s). En d18512 ambas agotan las 10.000 y tardan casi lo mismo (90,3 s frente a 82,3 s).

Con todo esto, en las instancias de hasta unos cientos de ciudades la búsqueda tabú es la que tiene mejor relación calidad-tiempo, porque por unos pocos segundos más que la búsqueda local divide su desviación entre 1,9 y 3,1. En las instancias grandes, con 10.000 iteraciones, ninguna de las búsquedas por trayectorias llega a aprovechar sus mecanismos y las diferencias entre ellas dependen sobre todo de la semilla. No hemos aplicado ningún test estadístico porque con solo tres ejecuciones por algoritmo no tendría potencia suficiente. Aun así, en las tres instancias pequeñas las diferencias entre la búsqueda local y la tabú son mucho mayores que la dispersión entre ejecuciones.

3.5. Análisis de la mejor solución

Si nos fijamos en la desviación, la mejor solución de todo el estudio es la de la búsqueda tabú en ch130, un recorrido de longitud 7.791,07 que es un 27,5 % más largo que el óptimo (6.110). Si comparamos con la mejor solución de la búsqueda local en la misma instancia (12.097,10; 98,0 %).

La búsqueda local acaba en un óptimo local para el intercambio de dos ciudades, y en estos recorridos suelen quedar aristas largas que atraviesan el plano y se cruzan entre ellas. Para quitar cada cruce habría que invertir un tramo entero del recorrido, y eso no se consigue con un solo intercambio que mejore el coste. El tabú sí puede salir de ahí porque acepta empeoramientos, y por eso sus recorridos van casi siempre de una ciudad a otra cercana. Pero como usa el mismo operador de intercambio, en 10.000 iteraciones no le da tiempo a quitar todos los cruces y saltos largos, y de ahí viene el 27,5 % que le falta. Una posible mejora sería cambiar el operador para que, en vez de intercambiar las dos ciudades, invierta el tramo del recorrido que hay entre ellas. Así se quitaría un cruce en un solo movimiento y se reduciría esta distancia.

Anexo

A. Carpeta de Logs

Enlace de descarga a la carpeta con los Logs: `https://drive.google.com/drive/folders/11J3Nz6esAcI5eahmU4_k4VhWtSjmZ47B?usp=sharing`

Dentro de la carpeta hay un subdirectorio por algoritmo (`gre`, `grea`, `b_local` y `tabu`). Los ficheros se llaman `<instancia>_<algoritmo>_<semilla>_ejecucion_<n>.log` y en ellos se registran los parámetros, la solución inicial, cada movimiento con su coste (de mejora o, en el tabú, también de empeoramiento), los reinicios por estancamiento, la solución final y un resumen con el coste y el tiempo de la ejecución.